Q13: How Do Models Represent and Store Knowledge, and Use It to Answer?

DLS 2026 — Module (Paradise)

October 2026

Learning objectives

After this module, a student should be able to (i) explain the key-value memory view of Transformer feed-forward layers, (ii) describe the causal-tracing evidence that mid-layer MLP modules mediate factual recall, (iii) state the linear representation hypothesis and the superposition account of polysemanticity, (iv) reason about locate-then-edit knowledge editing and its failure modes, and (v) distinguish knowledge stored in weights from knowledge supplied in context.

1 The question

A language model answers "The Eiffel Tower is located in the city of…" without any retrieval system, database, or explicit memory. Where does that fact live, in what format, and what computation retrieves it? This module surveys the best current answers. The honest summary up front: the evidence supports a *probabilistic* picture — facts are not stored in single neurons or single parameters, but there is convergent evidence, from several independent methods, that mid-layer feed-forward (MLP) modules are a major storage site, that recall is a partially attention-mediated lookup, and that the storage format is distributed over directions in activation space rather than aligned with individual neurons. All claims below are hedged; the field is young and the strongest results come from small- to mid-scale models (GPT-2 XL, GPT-J).

2 The storage site: FFN layers as key-value memories

Feed-forward (FFN) layers hold roughly two-thirds of a Transformer's parameters, yet received far less analysis than attention. Geva et al. [4] gave the influential reframing: an FFN layer

$$\mathrm{FFN}(x) = f(x\,K^\top)V \tag{1}$$

(where K, V are the two weight matrices and f the nonlinearity) is *structurally* a key-value memory — the same form as end-to-end memory networks [10]. Each row of K acts as a key: it correlates with human-interpretable input patterns (n-grams, themes, positions in the discourse). When the input x matches a pattern, the corresponding hidden unit activates, and the matching column of V — which induces a distribution over the output vocabulary — is added to the residual stream. Key findings from their analysis of BERT, RoBERTa, and GPT-2:

- Lower-layer keys match shallow patterns (trigrams, positions); higher-layer keys match semantically broader themes.

- Values complement keys: they raise the probability of tokens that plausibly *follow* the key pattern.
- A single FFN output is a *composition* (weighted sum) of many memories, refined layer by layer through the residual stream.

The view is a lens, not a theorem: FFN layers were not trained as explicit memory networks, and no one-to-one correspondence between a key row and a discrete fact is claimed. But the framing makes testable predictions — which the causal-tracing work below confirms in part.

3 The retrieval path: causal tracing and ROME

If FFN layers store associations, what is the evidence? Meng et al. [6] developed *causal tracing*: corrupt the input with noise, then restore (freeze to clean values) one internal state at a time and measure which restorations repair the model's factual prediction. The intervention identifies states that *causally mediate* the answer. Their findings on GPT-2 XL:

1. Factual recall localizes to **middle-layer MLP modules**, specifically while processing the **last token of the subject** (e.g., the final token of "The Eiffel Tower").
2. Late-layer **attention heads then move** (copy) the accumulated subject information to the final position where the object is predicted.
3. Storage and retrieval are thus *separable*: MLPs at the "early site" recall the association; attention at the "late site" routes it to the output.

To test the storage hypothesis, they introduced **ROME** (Rank-One Model Editing): treating the MLP's second-layer matrix as a linear associative memory, ROME solves for a rank-one update that maps a subject-derived key vector k_* to a new value vector v_* (encoding the new object) while minimally disturbing other stored associations (via a covariance-regularized closed form). ROME edits generalize across paraphrases yet stay specific to the edited subject — evidence that the mid-MLP storage picture is roughly right. The results are model- and fact-type-specific: causal traces are cleanest for crisp entity–attribute facts in autoregressive models, and later work found the causal-mediator picture does not fully transfer to all architectures and fact classes. Treat localization as a strong tendency, not a law.

4 The format: linear representations and superposition

Linear representation hypothesis. Features — the meaningful properties a network tracks — are represented as *directions* in activation space, with magnitude encoding intensity [3, 2]. A model's activations are (approximately) a linear superposition of feature vectors, $x \approx \sum_i a_i f_i$. This is what makes probing work: if information is linearly decodable from hidden states, a simple linear "diagnostic classifier" trained on frozen activations can read it out [5, 1]. But probing has a known caveat — high probing accuracy shows information is *accessible*, not that the model *uses* it; control tasks and causal interventions are the needed complement [1].

Superposition. If features were one-per-neuron, a d-dimensional layer could hold at most d features. Elhage et al. [2] showed with toy models that networks routinely store *more features than dimensions* by assigning them *non-orthogonal, nearly-collapsed* directions, exploiting the fact that in high dimensions many random vectors are almost mutually orthogonal. The cost is *interference*: reading out feature i picks up spurious contributions from feature j whenever they co-activate, and a nonlinearity (ReLU) is needed to filter the crosstalk. Three results from their toy models:

- Whether a feature is stored in superposition is governed by a **phase change** in feature sparsity and importance — past a threshold, the network jumps from storing a feature to collapsing it.
- Superposed features arrange into **geometric structures** (digons, triangles, higher-dimensional simplices and antipodal pairs), not random directions.
- **Polysemanticity** — one neuron responding to unrelated concepts — is the visible consequence: with more features than neurons, individual neurons must participate in many feature directions.

Superposition is what makes knowledge storage compact *and* what makes it hard to edit cleanly: a direction shared by many features cannot be changed without collateral effects. The demo in `code/superposition_demo.py` makes the capacity/interference trade concrete in NumPy.

5 Knowledge editing: locate, then edit — and its side effects

The storage picture suggests a maintenance strategy: locate where a fact lives, then patch it. ROME [6] edits one fact at a time; **MEMIT** [7] scales the idea, spreading a batch of edits across the range of mediating MLP layers, and reports thousands of simultaneous edits in GPT-J and GPT-NeoX while maintaining specificity and fluency. But editing has known side effects, which follow directly from the representation story:

- **Bleedover:** nearby subjects that share the edited key direction can inherit the new value — the covariance regularizer mitigates but does not eliminate this.
- **Degradation:** large or repeated updates drift the edited layer's behavior on unrelated text (fluency loss, perplexity increases).
- **Brittleness:** edited associations are directional ("$s \to o$" is stored separately from "$o \to s$"), and multi-hop inferences that *use* the fact may not follow it.
- **Reliability:** later evaluations found that popular editors — ROME and MEMIT included — can fail to preserve logical implications of edits, change unrelated behaviors, and even introduce fresh hallucinations; the "locate-then-edit" picture is a useful first approximation, not a solved mechanism.

The open problem is honest: we do not yet have a way to surgically update one fact in a superposed, distributed store with guarantees.

6 Knowledge in weights vs. knowledge in context

Everything above concerns knowledge baked into *parameters* during pre-training. Modern usage leans heavily on a second channel: knowledge supplied *in context* — documents pasted into the prompt, retrieved passages, conversation history. The distinction matters:

- **Capacity and update cost:** weights hold vast but static knowledge; updating requires editing or retraining. Context is tiny but instant: new information is "known" for the duration of the prompt.
- **Mechanism:** in-weights recall runs through the MLP-lookup path above; in-context use runs through attention reading the provided tokens — related to the induction-head machinery studied by Olsson et al. [8].
- **Failure modes:** in-weights knowledge fails by being absent, outdated, or distorted by superposition; in-context knowledge fails by retrieval errors, context overflow, and interference between passages.

Whether a model is "a knowledge base" [9] depends on the channel: parameteric knowledge behaves like a lossy, hard-to-update store; contextual knowledge behaves like a working memory with attention as the read mechanism. *Forward-link:* Q12 (*Why do models give confident answers even when wrong?*) examines the failure modes when these two channels conflict — a model with stale in-weights knowledge can override correct context, and vice versa.

7 Summary

- **Where:** mid-layer FFN/MLP modules are the best-supported storage site (key-value memory view; causal tracing), with attention routing information to the output.
- **What format:** features as directions (linear representation hypothesis), packed beyond capacity via superposition; polysemantic neurons are the symptom.
- **How to change it:** locate-then-edit (ROME/MEMIT) works as a first approximation but suffers bleedover, degradation, and brittleness — an open problem.
- **Which channel:** weights vs. context is the master distinction for reasoning about what a model "knows" at answer time.

References

[1] Yonatan Belinkov and James Glass. Analysis methods in neural language processing: A survey. *Transactions of the Association for Computational Linguistics*, 7:49–72, 2019.

[2] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. *Transformer Circuits Thread*, 2022.

[3] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.

[4] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5484–5495, 2021.

[5] Dieuwke Hupkes, Sara Veldhoen, and Willem Zuidema. Visualisation and "diagnostic classifiers" reveal how recurrent and recursive neural networks process hierarchical structure. *Journal of Artificial Intelligence Research*, 61:747–785, 2018.

[6] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. *Advances in Neural Information Processing Systems*, 35, 2022.

[7] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *The Eleventh International Conference on Learning Representations*, 2023.

[8] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Yuntao Askell, Anna Bai, Tom Chen, Nova Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Dario Amodei, Nicholas Joseph, and Chris Olah. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.

[9] Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, and Alexander Miller. Language models as knowledge bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.

[10] Sainbayar Sukhbaatar, Jason Weston, and Rob Fergus. End-to-end memory networks. *Advances in Neural Information Processing Systems*, 28, 2015.